
MOTOR DE INFERENCIA LÓGICA COMO SERVICIO

Documentación Técnica del Proyecto

Materia: Teoría de Lenguajes de Programación

Fecha: Mayo 2026

1. Introducción

El presente reporte técnico documenta el diseño, la arquitectura y la implementación del proyecto "Motor de Inferencia Lógica como Servicio". Este sistema tiene como objetivo principal exponer las capacidades de deducción de un motor Prolog a través de una API RESTful moderna, permitiendo a clientes externos realizar consultas lógicas sobre una base de conocimiento predefinida.

El proyecto surge de la necesidad de evaluar reglas de negocio complejas —específicamente enfocadas en el análisis de contratos y la aplicación de penalizaciones— separando la declaración de la lógica (qué es verdad) de la capa de transporte y enrutamiento (cómo se reciben y responden las peticiones). Al encapsular Prolog dentro de un servidor Node.js, se logra un servicio web interoperable que expone el poder de la programación lógica a cualquier aplicación capaz de consumir HTTP y JSON.

2. Paradigmas de Programación Utilizados

El proyecto integra tres paradigmas de programación fundamentales, cada uno cumpliendo un rol específico en la arquitectura del sistema.

2.1 Programación Lógica (Prolog / Tau Prolog)

El paradigma de programación lógica es el núcleo del sistema. En lugar de describir cómo resolver un problema paso a paso, se declaran hechos y reglas que describen las relaciones del dominio, y el motor de inferencia determina automáticamente las conclusiones.

Rol en el sistema

La base de conocimiento (`knowledge_base.pl`) contiene los hechos y reglas del dominio de contratos y penalizaciones. Tau Prolog actúa como el motor de inferencia que evalúa las consultas aplicando unificación y backtracking automático.

Características clave aplicadas

- **Hechos:** Verdades simples del dominio, como `expired(contract1)` o `has_penalty_clause(contract1)`.
- **Reglas:** Conclusiones derivadas mediante implicación lógica (`:-`), como `penalty_applicable(Contract) :- expired(Contract), has_penalty_clause(Contract), breach_reported(Contract)`.
- **Unificación:** El motor liga variables lógicas (`X`) automáticamente al evaluar consultas como `penalty_applicable(X)`.
- **Backtracking:** El motor explora todas las soluciones posibles de manera automática, retornando múltiples resultados cuando existen.
- **Negación por falla (`\+`):** Usada en `valid_contract` para verificar la ausencia de incumplimientos.

Ejemplo en el código:

```
% Regla compuesta con múltiples condiciones
penalty_applicable(Contract) :-
    contract(Contract),
    expired(Contract),
    has_penalty_clause(Contract),
    breach_reported(Contract).
```

2.2 Programación Funcional (JavaScript)

El paradigma funcional se aplica en la capa de transformación de datos del servidor Node.js. Se privilegia el uso de funciones puras, inmutabilidad y composición para el manejo de la entrada y salida del sistema.

Rol en el sistema

Las funciones funcionales se encargan de normalizar las consultas recibidas, formatear las respuestas JSON y convertir los términos Prolog a estructuras JavaScript legibles.

Características clave aplicadas

- **Funciones puras:** `normalizeQuery()`, `buildSuccessResponse()` y `buildErrorResponse()` no producen efectos secundarios y siempre retornan el mismo resultado para los mismos argumentos.
- **Transformación con map/filter:** Usados para iterar sobre los bindings Prolog y sobre las líneas de la base de conocimiento en el endpoint `/facts`.
- **Funciones de orden superior:** `termToJS()` aplica recursión funcional para transformar árboles de términos Prolog.

- **Spread operator y desestructuración:** Para construir objetos de respuesta de forma declarativa e inmutable.

Ejemplo en el código:

```
// Función pura: normaliza la consulta sin efectos secundarios
const normalizeQuery = (raw) => {
  const trimmed = String(raw).trim();
  return trimmed.endsWith('.') ? trimmed : `${trimmed}.`;
};
```

2.3 Programación Asíncrona (Node.js / async-await)

El modelo de ejecución de Node.js, basado en un event loop de un solo hilo con I/O no bloqueante, es aprovechado para manejar múltiples solicitudes concurrentes sin bloquear el servidor.

Rol en el sistema

Las operaciones de lectura del archivo de base de conocimiento y la ejecución del motor Prolog (que opera mediante callbacks internos) se envuelven en Promises y se consumen mediante `async/await`, permitiendo código asíncrono legible y mantenible.

Características clave aplicadas

- **async/await:** El handler del endpoint `/query` es una función `async` que aguarda la lectura del archivo y la resolución de la inferencia.
- **Promises:** `runPrologQuery()` encapsula la API de callbacks de Tau Prolog en una Promise, normalizando la interfaz asíncrona.
- **fs.promises:** Lectura no bloqueante del archivo `.pl` mediante `fs.readFile` (API de Promesas de Node.js).
- **Recursión asíncrona:** `getNextAnswer()` se llama recursivamente para iterar sobre las soluciones de Prolog, respetando el modelo de callbacks del motor.
- **Concurrencia:** Node.js puede atender múltiples solicitudes simultáneas gracias al event loop, sin necesidad de hilos adicionales.

Ejemplo en el código:

```
app.post('/query', async (req, res) => {
  // Operación asíncrona de I/O
  const kbSource = await fs.readFile(KB_PATH, 'utf8');

  // Operación asíncrona de inferencia lógica
  const answers = await runPrologQuery(kbSource, normalizedQuery, maxSols);
```

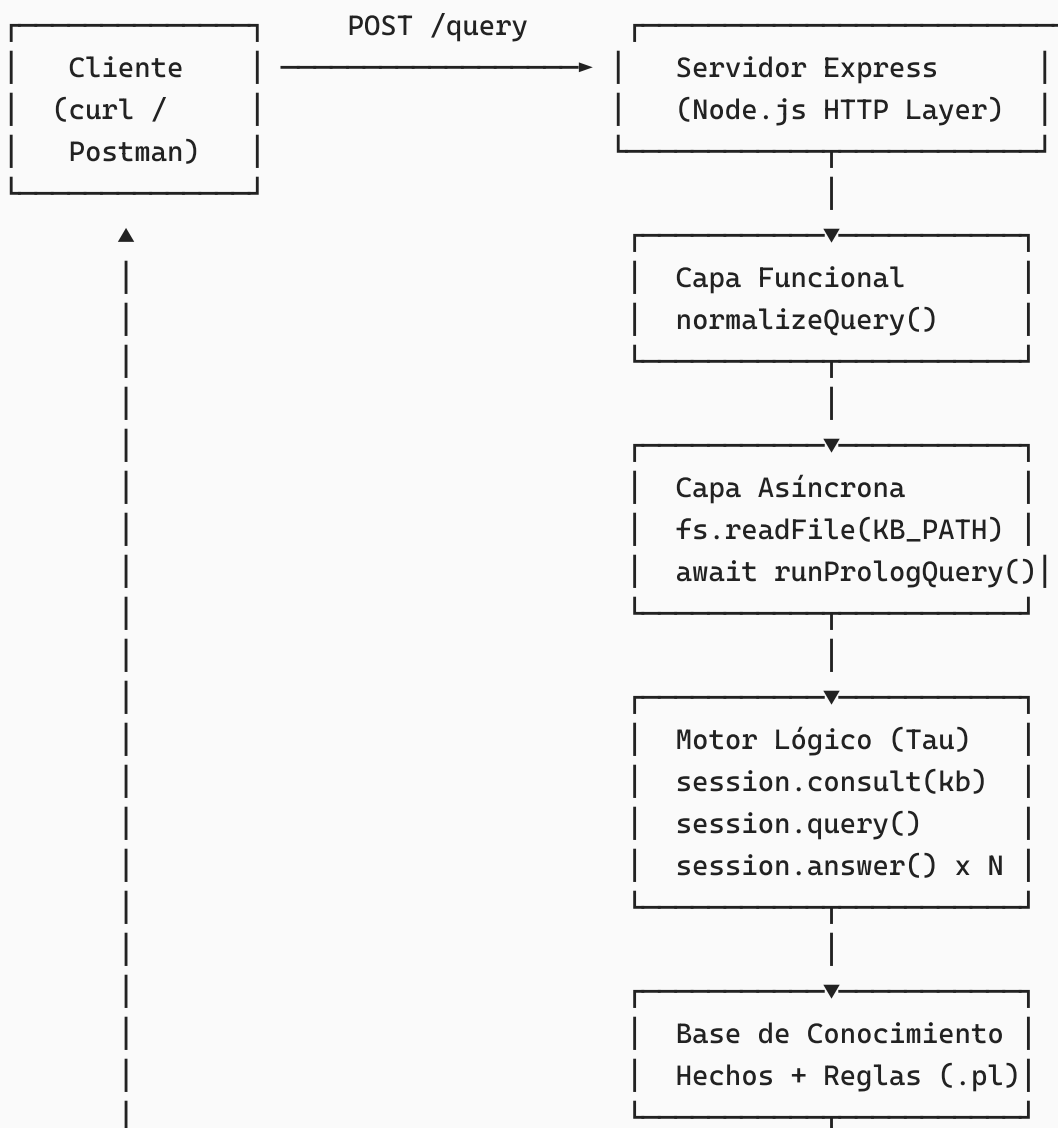
```
return res.json(buildSuccessResponse(normalizedQuery, answers));  
});
```

3. Arquitectura del Sistema

El sistema sigue una arquitectura de servicio de un solo módulo donde los tres paradigmas de programación colaboran en una cadena de procesamiento bien definida.

3.1 Diagrama de Flujo de una Solicitud

El siguiente diagrama describe el ciclo de vida de una consulta desde que el cliente la envía hasta que recibe la respuesta:



3.2 Componentes del Sistema

Componente	Tecnología	Responsabilidad
Servidor HTTP	Express.js	Recibir y enrutar solicitudes REST, enviar respuestas JSON
Capa Funcional	JavaScript (ES6+)	Normalización de entrada, formateo de salida, transformación de datos
Motor de Inferencia	Tau Prolog	Evaluar consultas lógicas mediante unificación y backtracking
Base de Conocimiento	Prolog (.pl)	Almacenar hechos y reglas del dominio de contratos
Capa Asíncrona	async/await + Promises	Manejar I/O no bloqueante y ejecución concurrente

3.3 Interacción entre Paradigmas

La arquitectura del sistema aprovecha las fortalezas de cada paradigma de forma complementaria:

- **El paradigma asíncrono** maneja la capa de transporte: recepción de solicitudes HTTP concurrentes, lectura del archivo de la base de conocimiento y entrega de respuestas sin bloquear el event loop.
- **El paradigma funcional** actúa como capa de transformación: funciones puras convierten la entrada del cliente a un formato adecuado para el motor lógico, y la salida del motor a un JSON estructurado para el cliente.
- **El paradigma lógico** implementa el dominio del negocio: toda la lógica de contratos, penalizaciones y reglas de inferencia se expresa de forma declarativa en Prolog, separando el "qué" del "cómo".

3.4 Decisiones de Diseño

- **Sin base de datos externa:** La base de conocimiento es un archivo `.pl` cargado en memoria por cada consulta, lo que simplifica el despliegue y facilita la comprensión del paradigma lógico.
- **Stateless por solicitud:** Cada petición crea su propia sesión Prolog, garantizando aislamiento y evitando efectos secundarios entre solicitudes.

- **Máximo de soluciones configurable:** El parámetro `max_solutions` permite al cliente controlar el límite de respuestas, evitando la generación infinita en consultas muy generales.
 - **Manejo de errores por capas:** Los errores de Prolog (sintaxis, inferencia) se distinguen de los errores de I/O, retornando mensajes descriptivos al cliente.
-

4. Ejemplo de Ejecución

A continuación, se detalla el flujo de una consulta real enviada al servicio utilizando Postman.

4.1 Petición del Cliente

Se realiza una petición de tipo `POST` al endpoint `/query`. En el cuerpo de la petición (JSON), se envía la consulta lógica para preguntar "A cuáles contratos aplica penalización".

Consulta: `penalty_applicable(X).`

NODE.JS + EXPRESS + TAU PROLOG

Motor de inferencia visualizable desde el navegador

Esta interfaz permite inspeccionar el servicio, consultar la base de conocimiento y probar reglas Prolog sin salir del navegador.

31

hechos detectados

7

consultas sugeridas

3000

puerto por defecto

Actualizar panel

Recargar hechos

ESTADO DEL SERVICIO

Salud del motor

● Servicio disponible

Motor: Tau Prolog

Timestamp: 2026-05-16T05:28:35.856Z

CONSULTAS

Probar una regla o hecho

Escribe una consulta Prolog. Si no agregas el punto final, el backend lo normaliza por ti.

penalty_applicable(contract1).

penalty_applicable(X).

at_risk(X).

valid_contract(X).

client_penalized(X).

applicable_penalty_amount(X, Y).

same_type(contract1, X).

Consulta Prolog

Ejemplo: penalty_applicable(X).

Máximo de soluciones

20

Ejecutar

Limpiar

BASE DE CONOCIMIENTO

Hechos cargados

Estos hechos se obtienen desde el endpoint /facts.

contract(contract1).

contract(contract2).

contract(contract3).

contract(contract4).

expired(contract1).

expired(contract3).

active(contract2).

active(contract4).

breach_reported(contract1).

CONSULTAS

Probar una regla o hecho

Escribe una consulta Prolog. Si no agregas el punto final, el backend lo normaliza por ti.

penalty_applicable(contract1).

penalty_applicable(X).

at_risk(X).

valid_contract(X).

client_penalized(X).

applicable_penalty_amount(X, Y).

same_type(contract1, X).

Consulta Prolog

penalty_applicable(X).

Máximo de soluciones

20

Ejecutar

Limpiar

BASE DE CONOCIMIENTO

Hechos cargados

Estos hechos se obtienen desde el endpoint /facts.

contract(contract1).

contract(contract2).

contract(contract3).

contract(contract4).

expired(contract1).

expired(contract3).

active(contract2).

active(contract4).

breach_reported(contract1).

breach_reported(contract3).

breach_reported(contract1).

breach_reported(contract3).

breach_reported(contract4).

RESPUESTA

Resultado de inferencia

```
{
  "status": "ok",
  "query": "penalty_applicable(X).",
  "satisfiable": true,
  "answers": [
    {
      "X": "contract1"
    },
    {
      "X": "contract3"
    }
  ],
  "count": 2
}
```

4.2 Procesamiento en el Servidor

El servidor recibe la petición y sigue los siguientes pasos:

1. Extrae y normaliza el texto (asegurando que termine con el punto final exigido por Prolog).
2. Lee el archivo `.pl` que contiene las reglas de los contratos de forma asíncrona.
3. El intérprete Tau Prolog evalúa la base de conocimiento cargada con la petición, realizando el proceso de backtracking para encontrar todos los valores de `X` que cumplen la condición.

4.3 Respuesta del Servicio

El sistema empaqueta la salida en un formato JSON estándar que indica si la consulta es satisficible, la cantidad de respuestas encontradas y las unificaciones de variables correspondientes.

The image shows the Prolog service interface on the left and its network response in a browser's developer tools on the right.

Prolog Service Interface:

- Variables:** `at_risk(X).`, `valid_contract(X).`, `client_penalized(X).`, `applicable_penalty_amount(X, Y).`, `same_type(contract1, X).`
- Consulta Prolog:** `penalty_applicable(X).`
- Máximo de soluciones:** 20
- Ejecutar** button
- Limpiar** button
- RESPUESTA:** Resultado de inferencia
- JSON Response:**

```
{  "status": "ok",  "query": "penalty_applicable(X).",  "satisfiable": true,  "answers": [    {      "X": "contract1"    },    {      "X": "contract3"    }  ],  "count": 2}
```
- Contract Rules:**
 - `contract(contract2).`
 - `contract(contract3).`
 - `contract(contract4).`
 - `expired(contract1).`
 - `expired(contract3).`
 - `active(contract2).`
 - `active(contract4).`
 - `breach_reported(contract1).`
 - `breach_reported(contract3).`

Network Response (Headers):

- General:**
 - Request URL: `http://localhost:3000/query`
 - Request Method: `POST`
 - Status Code: `200 OK`
 - Remote Address: `[::1]:3000`
 - Referrer Policy: `strict-origin-when-cross-origin`
- Response headers:**
 - Connection: `keep-alive`
 - Content-Length: `125`
 - Content-Type: `application/json; charset=utf-8`
 - Date: `Sat, 16 May 2026 05:50:18 GMT`
 - Etag: `W/"7d-aZx338CQRvcysfCHvSsY4gNdQ"`
 - Keep-Alive: `timeout=5`
 - X-Powered-By: `Express`
- Request Headers:**
 - Accept: `*/*`
 - Accept-Encoding: `gzip, deflate, br, zstd`
 - Accept-Language: `es-419;es;q=0.9;es-ES;q=0.8,en;q=0.7,en-GB;q=0.6,en-US;q=0.5;es-US;q=0.4;es-MX;q=0.3`
 - Connection: `keep-alive`
 - Content-Length: `53`
 - Content-Type: `application/json`

5. Conclusiones y Extensiones

Conclusiones

El desarrollo del Motor de Inferencia Lógica como Servicio ha demostrado que es plenamente viable y altamente productivo hacer convivir múltiples paradigmas de programación en un solo sistema. La programación lógica permite modelar el dominio (las reglas y contratos) de forma limpia y declarativa; la programación funcional aporta robustez e inmutabilidad en la transformación y serialización de los datos; y la programación asíncrona facilita un alto nivel de concurrencia al manejar la capa HTTP sin bloqueos de red.

Esta arquitectura modular y "stateless" (sin estado entre peticiones) asegura que el sistema sea escalable y que la lógica de negocio pueda modificarse sin necesidad de alterar el código

del servidor web.

Extensiones Propuestas

Para evolucionar el proyecto a futuro, se proponen las siguientes mejoras:

1. **Integración con Base de Datos:** Sustituir la lectura del archivo de texto plano (.pl) por un gestor de base de datos relacional o almacenamiento en la nube, permitiendo administrar hechos y reglas dinámicamente sin reiniciar el servidor.
2. **Interfaz Gráfica de Usuario (GUI):** Desarrollar un cliente visual (Frontend) donde el usuario final pueda construir consultas seleccionando opciones en lugar de escribir sintaxis Prolog y ver los resultados tabulados.
3. **Manejo de Sesiones Stateful:** Extender la API para permitir la aserción temporal de hechos lógicos durante una sesión específica, útil para simulaciones "what-if" por cliente.
4. **Validación de Entradas:** Implementar middlewares de seguridad para sanitizar las consultas y evitar inyecciones lógicas o consultas maliciosas que saturen el tiempo de cómputo del motor.